

Module 4: Fundamental Computing Algorithms

Module Overview

This module introduces the operations often carried out by most applications. They are the sorting and the searching operations. The examples are given together with their use cases with emphasis to the options as regards the growth of functions and sizes of input. Also, the section discusses numerical algorithms that use numerical analysis to solve mathematical problems. Furthermore, the module presents other abstract data structures that are widely used, such as hash tables and graphs, with how they are represented.

The module is broken down into two study units which are:

Unit 1: Numerical Algorithms

Unit 2: Sequential and Binary Search Algorithms; Binary Search Trees

Unit 3: Sorting Algorithms

Unit 4: Hash Tables; Graphs and its Representation

Study Unit 1: Numerical Algorithms

Expected Duration: # Week

Introduction

This unit presents the concept of numerical algorithms. Numerical algorithms are the subjects of numerical analysis, which use approximation for the problems of mathematical analysis. The study of numerical methods attempts at finding appropriate solutions of problems rather than the exact ones. In a nutshell, numerical analysis finds application in all fields of engineering and the physical sciences, and has recently been vital in biological sciences, social sciences, medicine, business, and arts. In particular, these algorithms are the ones used for designing shapes of cars and other automobiles, computing intensities for displaying graphics, animations, modelling and simulations, and supporting search engines, and many other practical problems of both research and commercial interests.

Nevertheless, there are many application areas for numerical algorithms in computing, such that it may be practically impossible to discuss many of them here. It is pertinent to state that some of the areas of application include, but not limited to solving linear equations, bandwidth reduction, signal processing and transmission, matrix multiplication, determinants and permanents, constrained and unconstrained optimisation, linear programming, random

number generation, factoring and optimality testing, discrete Fourier transformation, arbitrary-precision arithmetic, and many others.

However, in this courseware, for want of time, we shall limit our discussion of numerical algorithms to the areas listed below with the hope that students can grasp the concept of numerical algorithms and can always relate with this family of algorithms with relative ease.

Learning Outcomes for Study Unit 1

At the end of this study unit, you should be able to;

- 1.1 Define and use correctly the words in bold (SAQ1.1)
- 1.2 List and explain the challenges of numerical algorithms (SAQ1.2)
- 1.3 Enumerate the types of numerical methods and their associated algorithms (SAQ 1.3)
- 1.4 List the trends in numerical analysis (SAQ 1.4)

1.1 The Concept of Numerical Algorithms

Numerical analysis predates the invention of modern computers. Linear interpolation, which is a graphic method of curve fitting using linear polynomials to construct new data points within the range of discrete set of known data points, was already in use more than 2000 years ago. Early mathematicians embraced the subject of numerical analysis and thereby provided the foundation of current paradigms in numerical algorithms. These include; Newton's method, Lagrange interpolation polynomial, Gaussian elimination, and Euler's method. In fact, the origins are often connected with the 1947 publication by John Von Neumann and Herman Goldstine. However, another school of thought credits the development of numerical analysis to the 1912 work of Edmund Taylor Whittaker, who was a renowned British mathematician, physicist, and historian of science. He was a leading mathematical scholar of the early 20th century with significant contributions in the fields of mathematical physics and numerical analysis, including the theory of special functions, astronomy, celestial mechanics, history of physics, and digital signal processing.

For the purposes of understanding the goals of numerical algorithms, while it has been noted that the field of numerical analysis is the design and analysis of techniques that give approximate but accurate solutions to hard problems, the overall goals of numerical analysis can be seen in the following;

- a. Advanced numerical methods being essential in making numerical weather prediction.

- b. Computing the trajectory of a spacecraft which requires the accurate numerical solution of a system of ordinary differential equations.
- c. Car companies can improve the crash safety of vehicles by using computer simulations of crashes. Such simulations essentially consist of solving partial differential equations numerically.
- d. Hedge funds, which are usually private investment funds, use tools from fields of numerical analysis to attempt to calculate the value of stocks and derivatives more precisely than the market participants.
- e. Airlines use sophisticated optimisation algorithms to decide ticket prices, airplane and crew assignments and fuel needs. Historically, such algorithms were developed within the overlapping field of operations research.
- f. Insurance companies use numerical programs for actuarial analysis. Where actuarial analysis is the role of an Actuary who is a business professional with advanced statistical skills dealing with the measurement and management of risk and uncertainty. Importantly, actuaries provide assessments of financial security systems, with a focus on their complexity, their mathematics, and their mechanisms.

Numerical Analysis is the Mathematics branch responsible for designing effective ways to find numerical solutions to complex Mathematical problems. Most Mathematical problems from science and engineering are very complex and sometimes cannot be solved directly. Therefore, measuring a complex Mathematical problem is very important to make it easier to solve. Due to the great advances in computational technology, numeracy has become very popular and is a modern tool for scientists and engineers. As a result many software programs are being developed such as Matlab, Mathematica, Maple, etc, that solve the most difficult problems in an effective and simple way. These software systems contain functions that use standard numeric methods, in which the user can bypass the required parameters and obtain the results in a single command without knowing the numerical details.

The Numerical Analysis method is mainly used in the area of Mathematics and Computer Science that creates, analyzes, and implements algorithms for solving numerical problems of continuous Mathematics. Such types of problems generally originate from real-world applications of algebra, geometry and calculus, and they also involve variables that vary continuously. These problems occur throughout the natural sciences, social sciences, engineering, medicine, and the field of business. With the introduction of Numerical Analysis

during the past half-century, the growth in power and availability of digital computers has led to the increasing use of realistic Mathematical models in science and engineering. Here we will learn more about numerical method and analysis of numerical methods.

In-text Question: Two computer software systems that implements numerical analysis are:

Solution: Matlab, Mathematica

1.2 Challenges of Numerical Analysis

- i. **Accuracy:** Every numerical method introduces errors. It may be due to the use of the proper Mathematical process or due to accurate representation and change of numbers on the computer.
- ii. **Efficiency:** Another consideration in choosing a numerical method for a mathematical model solution efficiency, which means the amount of effort required by both people and computers to use the method.
- iii. **Numerical instability:** Another problem presented by a numerical method is numerical instability. Errors included in the calculation, from any source, increase in different ways. In some cases, these errors are usually rapid, resulting in catastrophic results.

1.3 Numerical Methods

Numerical methods are techniques that are used to approximate Mathematical procedures. We need approximations because we either cannot solve the procedure analytically or because the analytical method is intractable (an example is solving a set of a thousand simultaneous linear equations for a thousand unknowns).

1.3.1 Different Types of Numerical Methods

The numerical analysts and mathematicians have a variety of tools that they use to develop numerical methods for solving Mathematical problems. The most important idea, mentioned earlier, that cuts across all sorts of mathematical problems is that of changing a given problem with a 'near problem' that can be easily solved. There are other ideas that differ on the type of Mathematical problem solved.

An introduction to numerical methods for solving common division problems is contained in the following:

- i. Euler method - the most basic way to solve ODE
- ii. Clear and vague methods - vague methods need to solve the problem in every step
- iii. The Euler Back Road - the obvious variation of the Euler method
- iv. Trapezoidal law - the direct method of the second system
- v. Runge-Kutta Methods - one of the two main categories of problems of the first value.

1.3.2 Numerical Computation

The term “numerical computation” means to use computers for solving problems involving real numbers. In this process of problem-solving, we can distinguish several more or less distinct phases. The first phase is formulation. While formulating a mathematical model of a physical situation, scientists should take into account the fact that they expect to solve a problem on a computer. Therefore they will provide for specific objectives, proper input data, adequate checks, and for the type and amount of output.

Once a problem has been formulated, then the numerical methods, together with preliminary error analysis, must be devised for solving the problem. A numerical method that can be used to solve a problem is called an algorithm. An algorithm is a complete and unambiguous set of procedures that are used to find the solution to a mathematical problem. The selection or construction of appropriate algorithms is done with the help of Numerical Analysis. We have to decide on a specific algorithm or set of algorithms for solving the problem, numerical analysts should also consider all the sources of error that may affect the results. They should consider how much accuracy is required. To estimate the magnitude of the round-off and discretization errors, and determine an appropriate step size or the number of iterations required.

The programmer should transform the suggested algorithm into a set of unambiguous instruction that is followed step-by-step by the computer. The flow chart is the first step in this procedure. A flow chart is simply set of procedures, which are usually written in logical block form, which the computer will follow. The complexity of the flow will depend upon the complexity of the problem and the amount of detail included. However, it should be possible for someone else other than the programmer to follow the flow of information from

the chart. The flow chart is an effective aid to the programmer. They must translate its major functions into a program. And, at the same time, it is an effective means of communication to others who wish to understand what the program does, which is the central essence of algorithms in general.

1.3.3 Algorithms for Numerical Methods

i. Newton method

Some calculations cannot be solved using algebra or other mathematical methods. For this we need to use numerical methods. Newton's method is one such method and allows us to calculate the solution of $f(x) = 0$.

ii. Simpson Law

The other important ones cannot be assessed in terms of integration rules or basic functions. Simpson's law is a numerical method that calculates the numerical value of a direct combination.

iii. Trapezoidal law

A trapezoidal rule is a numerical method that calculates the numerical value of a direct combination. The other important ones cannot be assessed in terms of integration rules or basic functions. In mathematics, trapezoidal law or trapezium law, is the most important measure of direct equity in Numerical Analysis. Trapezoidal law is a coupling law used to calculate the area under a curve by dividing the curve into a small trapezoid. The combination of all the small trapezoid areas will provide space under the curve. Let's understand the trapezoidal law formula and its evidence using examples in future sections.

1.4 Trends in Numerical Analysis

The following are recent trends and application of numerical analysis:

a. High-performance computing

One of the main drivers of numerical analysis software and tools is the need for high-performance computing (HPC), which enables the solution of large-scale and complex problems that require massive amounts of computation and data. HPC involves the use of parallel and distributed architectures, such as clusters, supercomputers, cloud platforms, and

edge devices, to achieve high speed, scalability, and efficiency. However, HPC also introduces some challenges for numerical analysis software and tools, such as managing concurrency, communication, synchronization, load balancing, fault tolerance, security, and energy consumption. Therefore, numerical analysis software and tools need to be designed and optimized for HPC environments, using appropriate programming models, libraries, frameworks, and standards.

In-text Question: The process of using a computer to solve problems involving real numbers is known as?

Solution: Numerical methods

b. Machine learning and artificial intelligence

Another major trend in numerical analysis software and tools is the integration of machine learning and artificial intelligence (ML/AI) techniques, which enable the extraction of patterns, insights, and predictions from large and complex data sets. ML/AI techniques rely on numerical analysis algorithms for data preprocessing, feature engineering, model training, evaluation, and inference. Moreover, numerical analysis algorithms can also benefit from ML/AI techniques, such as using neural networks to approximate functions, optimize parameters, or learn representations. However, ML/AI also poses some challenges for numerical analysis software and tools, such as ensuring accuracy, reliability, robustness, interpretability, and fairness of the results, as well as dealing with uncertainty, noise, and adversarial attacks. Therefore, numerical analysis software and tools need to incorporate ML/AI methods and principles, using appropriate tools, platforms, and frameworks.

c. Reproducibility and verification

A crucial aspect of numerical analysis software and tools is the reproducibility and verification of the results, which ensures the validity, credibility, and usability of the solutions. Reproducibility and verification involve documenting, sharing, and testing the code, data, parameters, and methods used to obtain the results, as well as comparing and validating them against theoretical or empirical benchmarks. However, reproducibility and verification also face some challenges in numerical analysis software and tools, such as dealing with the complexity, variability, and heterogeneity of the software and tools, the dependence on external factors, such as hardware, software, and network configurations, and

the lack of standards, guidelines, and incentives for reproducible research. Therefore, numerical analysis software and tools need to adopt and promote best practices and tools for reproducibility and verification, such as using version control, documentation, testing, packaging, archiving, sharing, and citation systems.

1.5 Summary of Study Unit 1

In this unit, you have learned the following;

1. Numerical analysis is the design and analysis of techniques that give approximate but accurate solutions to hard problems.
2. Numerical Analysis is the Mathematics branch responsible for designing effective ways to find numerical solutions to complex Mathematical problems
3. Advanced numerical methods being essential in making numerical weather prediction.
4. The challenges of numerical analysis include accuracy, efficiency and numerical instability.
5. The different types of numerical methods include; Euler method, Clear and vague, the Euler Back Road, Trapezoidal law, Runge-Kutta Methods
6. Numerical computation means to use computers for solving problems involving real numbers.
7. Some notable algorithms for numerical methods are Newton law, Simpson law and Trapezoidal law.
8. The trends in numerical analysis are high performance computing, machine learning, and artificial intelligence and Reproducibility and verification.

1.6 Self-Assessment Questions (SAQs) for Study Unit 1

Now that you have completed this study session, you can check to see how well you have achieved its Learning Outcomes by answering the following questions.

SAQ 1.1 (Tests learning outcome 1.1)

1. What is do you understand by the term "Numerical Analysis"?

SAQ 1.2 (Tests learning outcome 1.2)

1. What are the challenges of numerical analysis?

SAQ 1.3 (Tests learning outcome 1.3)

1. Techniques that are used to approximate Mathematical procedures are called?
2. Identify the odd ones in the following; Euler method, Clear and vague, the Euler Back Road, Trapezoidal law, Euler-Range methods, Runge-Kutta Methods and Euler Rapid approach.

SAQ 1.4 (Tests learning outcome 1.4)

1. Two trends in numerical analysis are and

References for further reading

Eiselt, Horst A et al. Integer Programming and Network Models. Springer, 2011.

Introduction to Algorithms (Cormen, Leiserson, Rivest, and Stein) 2001, Chapter 16 "Greedy Algorithms".

Eiselt, Horst A et al. Integer Programming and Network Models. Springer, 2011.

J.H. Holland (1975) *Adaptation in Natural and Artificial Systems*, University of Michigan Press, Ann Arbor, Michigan; re-issued by MIT Press (1992).

Optimal design of heat exchanger networks, Editor(s): Wilfried Roetzel, Xing Luo, Dezhen Chen, Design and Operation of Heat Exchangers and their Networks, Academic Press, 2020, Pages 231-317, ISBN 9780128178942, <https://doi.org/10.1016/B978-0-12-817894-2.00006-6>.

Wang FS., Chen LH. (2013) Genetic Algorithms. In: Dubitzky W., Wolkenhauer O., Cho KH., Yokota H. (eds) *Encyclopedia of Systems Biology*. Springer, New York, NY. https://doi.org/10.1007/978-1-4419-9863-7_412

Fred Glover (1986). "Future Paths for Integer Programming and Links to Artificial Intelligence". *Computers and Operations Research*. **13** (5): 533–549, [https://doi.org/10.1016/0305-0548\(86\)90048-1](https://doi.org/10.1016/0305-0548(86)90048-1)

Optimization of Preventive Maintenance Program for Imaging Equipment in Hospitals, Editor(s): Zdravko Kravanja, Miloš Bogataj, Computer-Aided Chemical Engineering, Elsevier, Volume 38, 2016, Pages 1833-1838, ISSN 1570-7946, ISBN 9780444634283, <https://doi.org/10.1016/B978-0-444-63428-3.50310-6>.

Glover, Fred, and Gary A Kochenberger. Handbook Of Metaheuristics. Kluwer Academic Publishers, 2003.

Kirkpatrick, S., Gelatt, C., & Vecchi, M. (1983). Optimization by Simulated Annealing. *Science*, 220(4598), 671-680. Retrieved November 25, 2020, from <http://www.jstor.org/stable/1690046>

Brief review of static optimization methods, Editor(s): Stanisław Sieniutycz, Jacek Jeżowski, Energy Optimization in Process Systems and Fuel Cells (Third Edition), Elsevier, 2018, Pages 1-41, ISBN 9780081025574, <https://doi.org/10.1016/B978-0-08-102557-4.00001-3>.

NIU, M., WAN, C. & Xu, Z. A review on applications of heuristic optimization algorithms for optimal power flow in modern power systems. *J. Mod. Power Syst. Clean Energy* 2, 289–297 (2014), <https://doi.org/10.1007/s40565-014-0089-4>

J. L. Wang, Y. H. Lin and M. D. Lin, "Application of heuristic algorithms on groundwater pumping source identification problems," 2015 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM), Singapore, 2015, pp. 858-862, <https://doi.org/10.1109/IEEM.2015.7385770>.

Matott, L. Shawn, et al. "Application of Heuristic Optimization Techniques and Algorithm Tuning to Multilayered Sorptive Barrier Design." *Environmental Science & Technology*, vol. 40, no. 20, 2006, pp. 6354–6360., <https://doi.org/10.1021/es052560+>.

Larranaga P, Calvo B, Santana R, Bielza C, Galdiano J, Inza I, Lozano JA, Armananzas R, Santafe G, Perez A, Robles V (2006) Machine learning in bioinformatics. *Brief Bioinform* 7(1):86–112

Study Unit 2: Sequential and Binary Search Algorithms, Binary Search Trees

Expected Duration: # week

Introduction

In this unit, we shall be looking at basic search algorithms which are used to retrieve information during computing.

Learning Outcomes for Study Unit 2

At the end of this study unit, you should be able to;

- 2.1 Define and use correctly the words in bold (SAQ2.1)
- 2.2 Write and explain the sequential search algorithm (SAQ2.2)
- 2.3. Write and explain the Binary search algorithm (SAQ 2.3)
- 2.4. List and explain the attributes and types of a Binary Search Tree (SAQ 2.4)

2.1 The concept of Searching in Computing

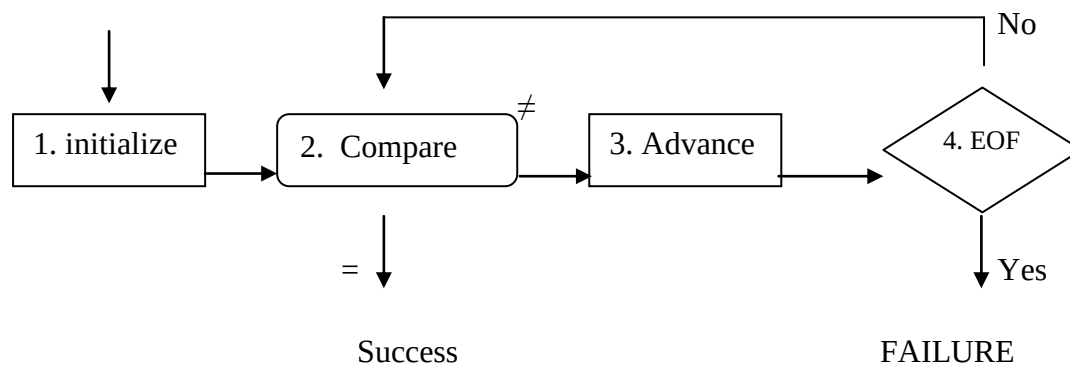
Searching is another important and fundamental operation in computing. A searching algorithm operates by examining a set of data elements to find a designated (a given identification) target element. For example, in a numerical application, we might want to find $f(x)$, given x and a table of the values of f . In a non numerical application, we might want to find the English translation of a given French/Greek/Russian word. For searching purposes, we suppose that a set of N records has been stored, with distinct N keys that uniquely identify the elements of the records. The collection of records is called a file or a table. A large file or group of files is frequently called a database. Algorithms for searching are presented with an argument, say K , and the problem is to find which record has K as its key. The result of a search operation will usually be any of the two possibilities: successful, having located the record containing K ; or unsuccessful having determined that K is nowhere to be found. Searching involves the use of three fundamental queries, existence, retrieval, and associative lookup. The classification of searching algorithms is dependent on their operational nature: Internal or external; static or dynamic; based on comparisons between keys or on digital properties of keys; those that work with actual keys or those that work with transformed keys.

Examples of searching algorithm include; the sequential search (brute force), binary search, and hash-based searches, which can be based on arithmetic transformations of the actual keys. We here attempt to discuss only sequential search and binary search owing to the constraints of time. Further studies on hash-based searches can be made with the support of ideas from the two treated.

2.2 Sequential Search: This is also called linear search. “Begin at the Beginning”, and go on till you find the right key; then stop. Considering a target element, say of value t in some collection C ; there are two conditions which terminate the search.

- a. The element is found, i.e. $a_i = t$ where $t \in C$.
- b. The entire array has been scanned, and no match was found.

This can be illustrated with the following flow diagram:



- g. Input: The input consists of a collection, C , of $n \geq 0$ elements and a target item, t , being sought.
- h. Output: The output is a return of true if t belongs to C , and false otherwise.

2.2.1 The Sequential Search Algorithm

Search (A, t)

1. for $I = 0$ to $n - 1$ do
2. If $(A[I] = t)$ then
3. return true
4. else return false
5. End

In-text Question: Another name for sequential search is?

Solution: Linear search

2.3 Binary Search

Binary search, a popular divide-and-conquer searching strategy, is known to deliver better performance than sequential search over a table with a seemingly large amount of entries.

It operates by first of all dividing the entries into halves and determining the middle entry. If the middle entry corresponds to the target element, it terminates successfully; else it goes to determine the corresponding half with the greater probability of containing the target element. It terminates with failure, if the target element cannot be found in both halves. This drastically reduces the search time when compared to searching sequentially.

Binary search can be illustrated with the following flow diagram;

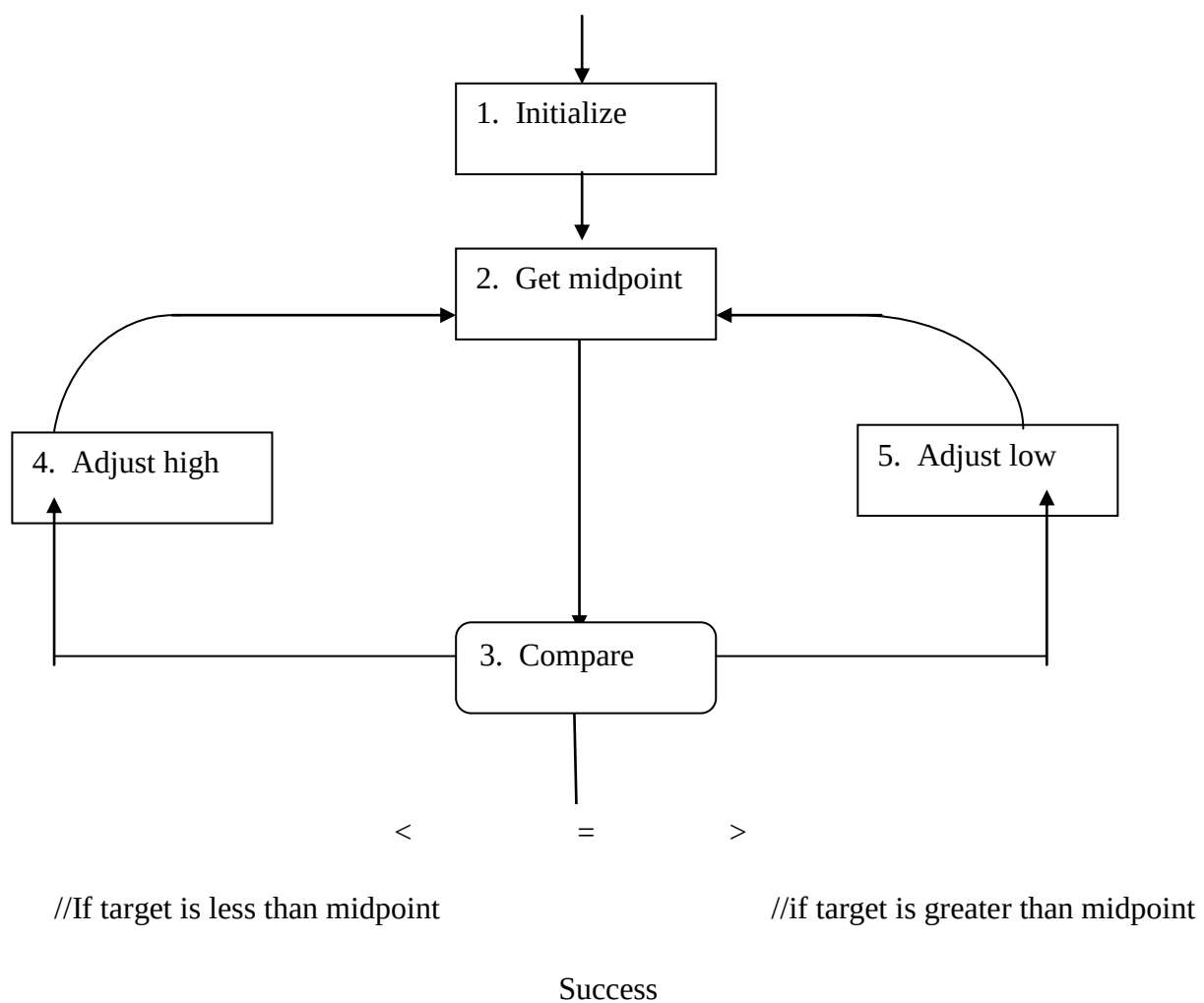


Figure: Binary Search

Example 1

Let's look at the following example to understand the binary search working

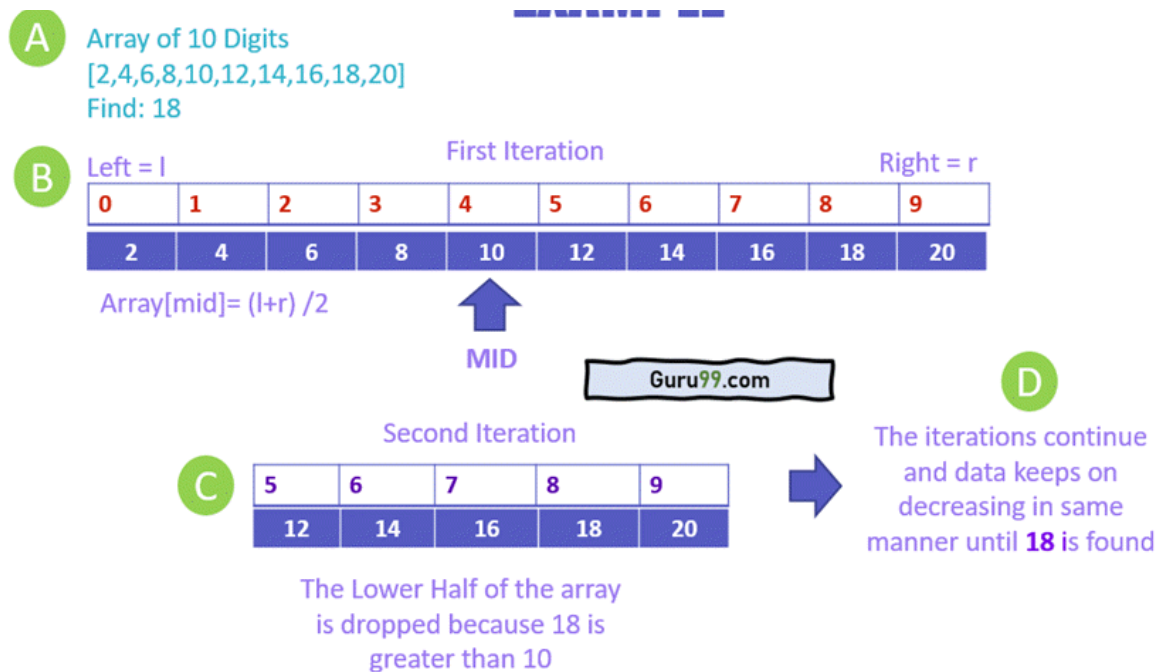


Figure: Binary Search Example

- You have an array of sorted values ranging from 2 to 20 and need to locate 18.
- The average of the lower and upper limits is $(l + r) / 2 = 4$. The value being searched is greater than the mid which is 4.
- The array values less than the mid are dropped from search and values greater than the mid-value 4 are searched.
- This is a recurrent dividing process until the actual item to be searched is found.

Binary search, therefore, is assumed to sort the elements in a collection in advance of query.

Thus, Binary search is useful for searching ordered set:

$$A_k : 1 \leq k < N \text{ such that } a_{k-1} \leq a_k$$

- It is important to note that the expected number of comparisons in binary search is $N/2$, while that of sequential search is N . (Refer to the section on efficiency for further details of their performance evaluation)
- Input: The input to binary search is an indexed collection, say A of elements. Each element $A[i]$ has a key value K_i that uniquely identifies the elements. These keys are totally ordered, as stated, which means that given two keys K_i and K_j we have that K_i

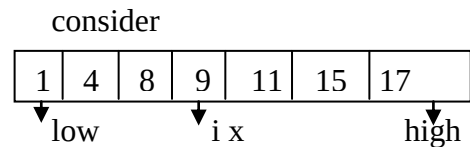
$< K_j$, $K_i = K_j$, or $K_i > K_j$. The data structure is constructed and preserves the ordering of the keys.

k. Output: the output is either true or false.

2.3.1 Binary Search Algorithm

Search (A, t)

- a. low = 0
- b. high = n - 1
- c. while (low ≤ high) do
- d. ix = (low + high)/2 //using floor/ceiling
- e. If (t = A[ix]) then
- f. return true
- g. else if (t < A[ix]) then
- h. high = ix - 1
- i. else low = ix + 1
- j. return false
- k. End



Comment: we assume even numbered array for the above algorithm to be implementable, else the floor or ceiling function can be used to deal with odd numbered array. There are other suggestions in literature for computing the mid point, such as those of $ix = (R-L)/2+L$; in order to fix the bugs associated with the actual implementation of the algorithm, as a matter of the problem of representing the finite bounds of the chosen integer types.

2.3.2 Why Do We Need Binary Search?

The following reasons make the binary search a better choice to be used as a search algorithm:

- i. Binary search works efficiently on sorted data no matter the size of the data
- ii. Instead of performing the search by going through the data in a sequence, the binary algorithm randomly accesses the data to find the required element. This makes the search cycles shorter and more accurate.
- iii. Binary search performs comparisons of the sorted data based on an ordering principle than using equality comparisons, which are slower and mostly inaccurate.

- iv. After every cycle of search, the algorithm divides the size of the array into half hence in the next iteration it will work only in the remaining half of the array

2.4 Binary Search Tree (BST) with Example

The binary search tree is an advanced algorithm used for analyzing the node, its left and right branches, which are modeled in a tree structure and returning the value. The BST is devised on the architecture of a basic binary search algorithm; hence it enables faster lookups, insertions, and removals of nodes. This makes the program really fast and accurate.

2.4.1 Attributes of Binary Search Tree

A BST is made of multiple nodes and consists of the following attributes:

- i. Nodes of the tree are represented in a parent-child relationship
- ii. Each parent node can have zero child nodes or a maximum of two subnodes or subtrees on the left and right sides.
- iii. Every sub-tree, also known as a binary search tree, has sub-branches on the right and left of themselves.
- iv. All the nodes are linked with key-value pairs.
- v. The keys of the nodes present on the left subtree are smaller than the keys of their parent node
- vi. Similarly, The left subtree nodes' keys have lesser values than their parent node's keys.

CONCEPT DIAGRAM

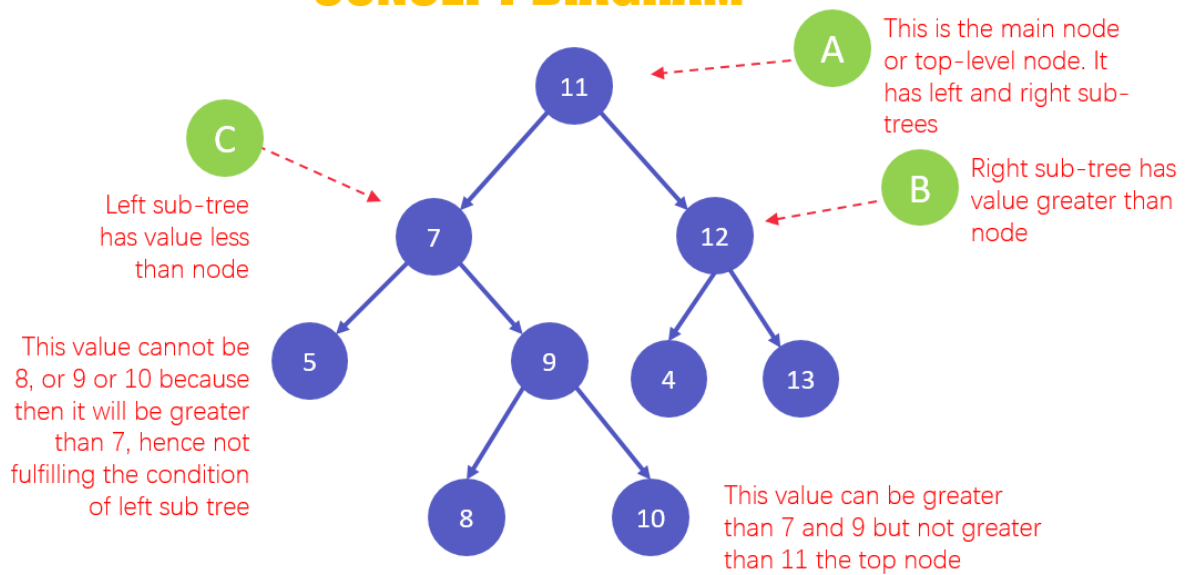


Figure: Binary Search Tree

- A. There is the main node or parent level 11. Under it, there are left and right nodes/branches with their own key values
- B. The right sub-tree has key values greater than the parent node
- C. The left sub-tree has values than the parent node

2.4.2 Why do we need a Binary Search Tree?

- i. The two major factors that make binary search tree an optimum solution to any real-world problems are Speed and Accuracy.
- ii. Due to the fact that the binary search is in a branch-like format with parent-child relations, the algorithm knows in which location of the tree the elements need to be searched. This decreases the number of key-value comparisons the program has to make to locate the desired element.
- iii. Additionally, in case the element to be searched greater or less than the parent node, the node knows which tree side to search for. The reason is, the left sub-tree is always lesser than the parent node, and the right sub-tree has values always equal to or greater than the parent node.
- iv. BST is commonly utilized to implement complex searches, robust game logics, auto-complete activities, and graphics.
- v. The algorithm efficiently supports operations like search, insert, and delete.

2.4.3 Types of Binary Trees

Three kinds of binary trees are:

- i. Complete binary tree: All the levels in the trees are full of last level's possible exceptions. Similarly, all the nodes are full, directing the far left.
- ii. Full binary tree: All the nodes have 2 child nodes except the leaf.
- iii. Balanced or Perfect binary tree: In the tree, all the nodes have two children. Besides, there is the same level of each subnode.

2.4.4 Operations on a Binary Tree

The tree always has a root node and further child nodes, whether on the left or right. The algorithm performs all the operations by comparing values with the root and its further child nodes in the left or right sub-tree accordingly.

Depends upon the element to be inserted, searched, or deleted, after the comparison, the algorithm can easily drop the left or right subtree of the root node.

BST primarily offers the following three types of operations for your usage:

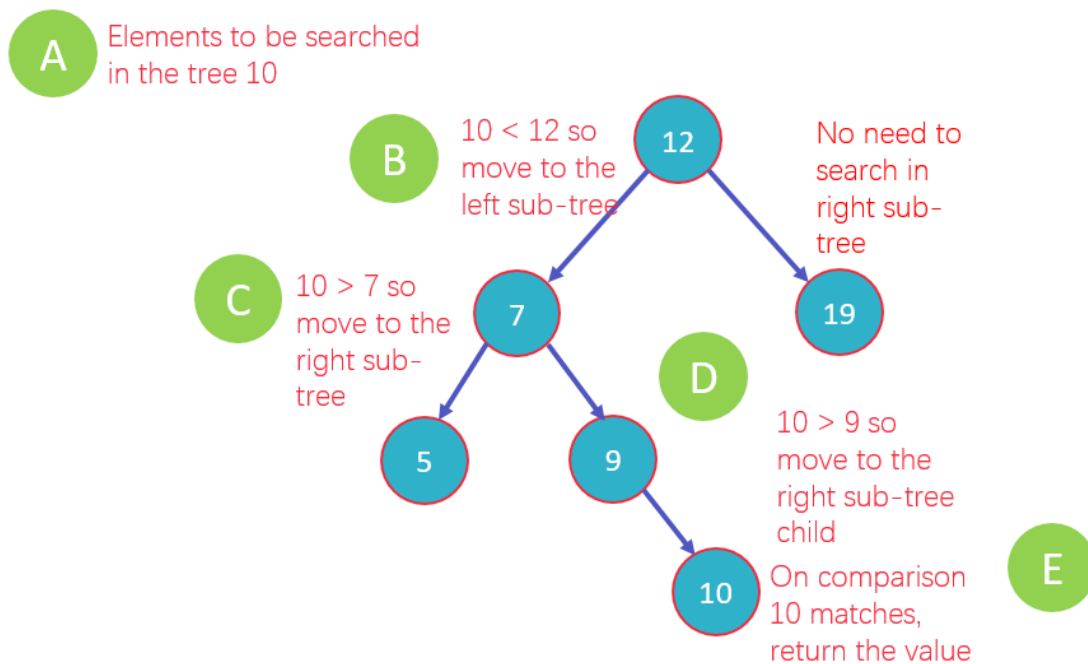
- a. Search: searches the element from the binary tree
- b. Insert: adds an element to the binary tree
- c. Delete: delete the element from a binary tree

Each operation has its own structure and method of execution/analysis, but the most complex of all is the Delete operation.

a. Search Operation

Always initiate analyzing tree at the root node and then move further to either the right or left subtree of the root node depending upon the element to be located is either less or greater than the root.

Search Operation



- A. The element to be searched is 10
- B. Compare the element with the root node 12, $10 < 12$, hence you move to the left subtree. No need to analyze the right-subtree
- C. Now compare 10 with node 7, $10 > 7$, so move to the right-subtree
- D. Then compare 10 with the next node, which is 9, $10 > 9$, look in the right subtree child
- E. 10 matches with the value in the node, $10 = 10$, return the value to the user.

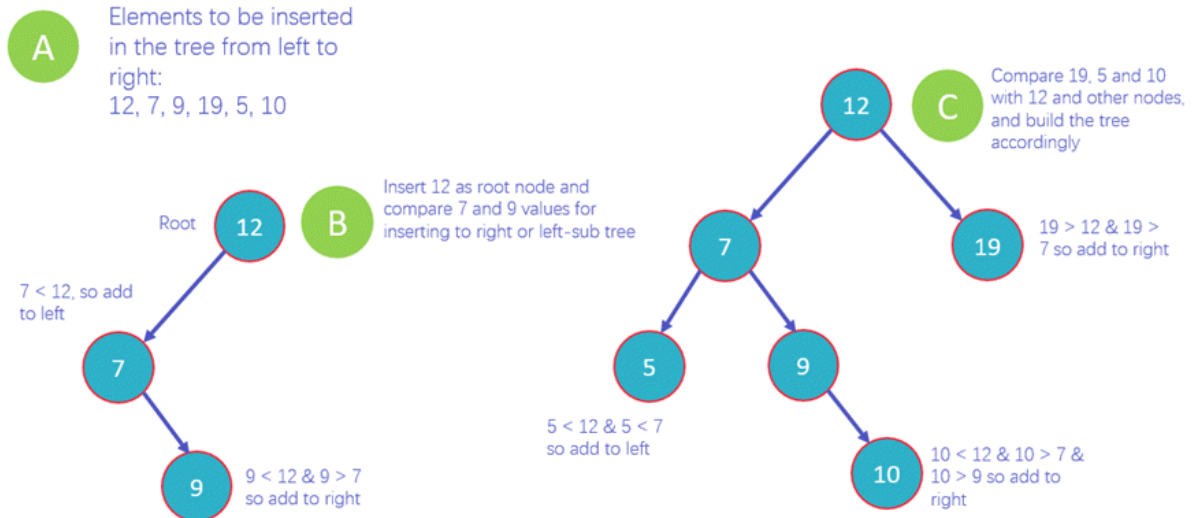
Pseudo Code for Searching in BST:

```
search(element, root)
    if !root
        return -1
    if root.value == element
        return 1
    if root.value < element
        search(element, root.right)
    else
        search(element, root.left)
```

b. Insert Operation

This is a very straight forward operation. First, the root node is inserted, then the next value is compared with the root node. If the value is greater than root, it is added to the right subtree, and if it is lesser than the root, it is added to the left subtree.

Insert Operation



- There is a list of 6 elements that need to be inserted in a BST in order from left to right
- Insert 12 as the root node and compare next values 7 and 9 for inserting accordingly into the right and left subtree
- Compare the remaining values 19, 5, and 10 with the root node 12 and place them accordingly. $19 > 12$ place it as the right child of 12, $5 < 12$ & $5 < 7$, hence place it as left child of 7. Now compare 10, 10 is < 12 & $10 > 7$ & $10 > 9$, place 10 as right subtree of 9.

In-text Question: Three major operations on a binary search tree are?

Solution: Search, insert, and delete operations

Pseudocode for Inserting a Node in BST:

```

insert (element, root)
    Node x = root
    Node y = NULL
    while x:
        y = x
        if x.value < element.value
            x = x.right
        else
            x = x.left
        if y.value < element
            y.right = element
        else
            y.left = element
    
```

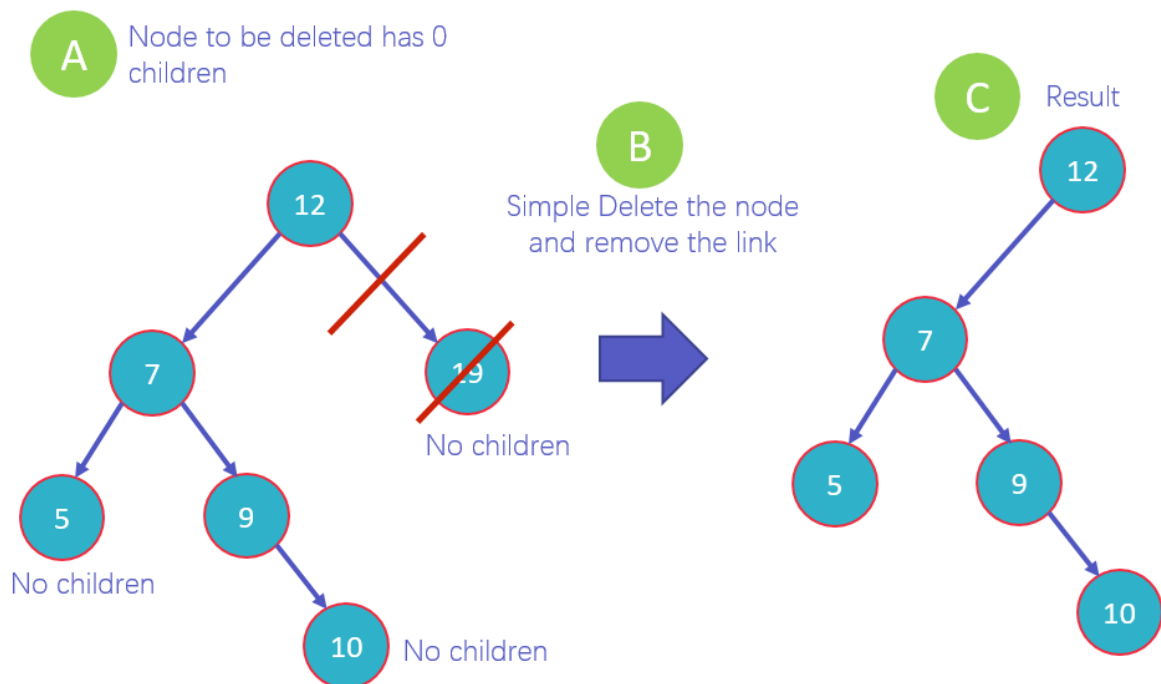
Delete Operations

For deleting a node from a BST, there are some cases, i.e. deleting a root or deleting a leaf node. Also, after deleting a root, we need to think about the root node.

Say we want to delete a leaf node, we can just delete it, but if we want to delete a root, we need to replace the root's value with another node. Let's take the following example:

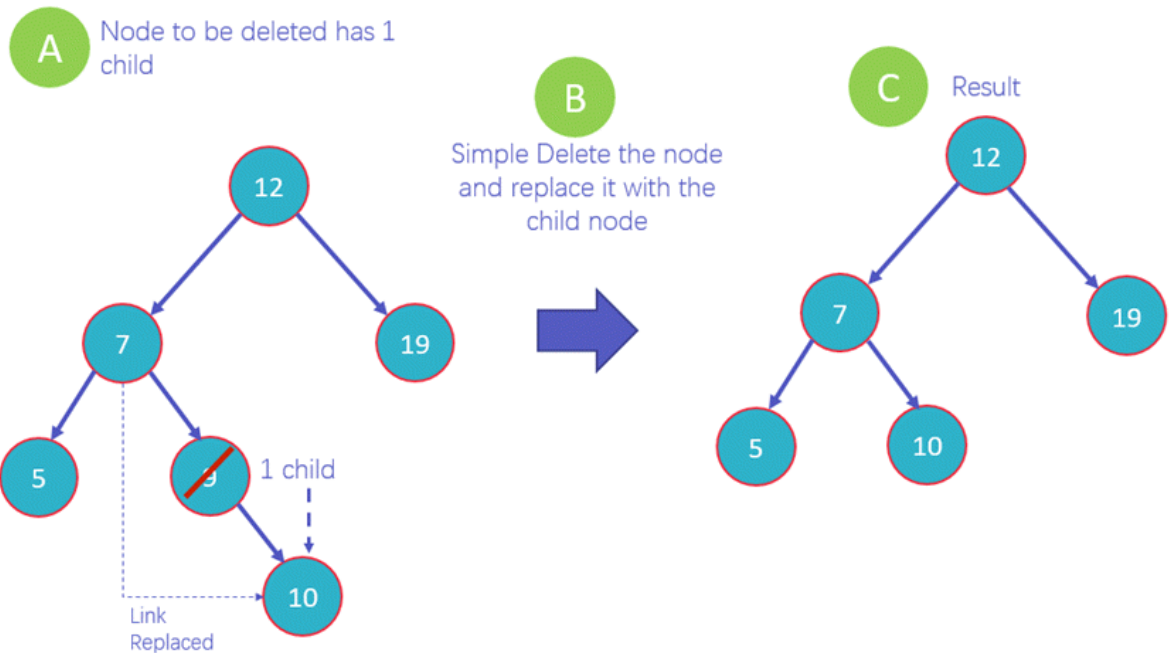
- i. Case 1- Node with zero children: this is the easiest situation; you just need to delete the node which has no further children on the right or left.
- ii. Case 2 – Node with one child: once you delete the node, simply connect its child node with the parent node of the deleted value.
- iii. Case 3 Node with two children: this is the most difficult situation, and it works on the following two rules
 - 3a – In Order Predecessor: you need to delete the node with two children and replace it with the largest value on the left-subtree of the deleted node
 - 3b – In Order Successor: you need to delete the node with two children and replace it with the largest value on the right-subtree of the deleted node

Delete Operation – Case 1



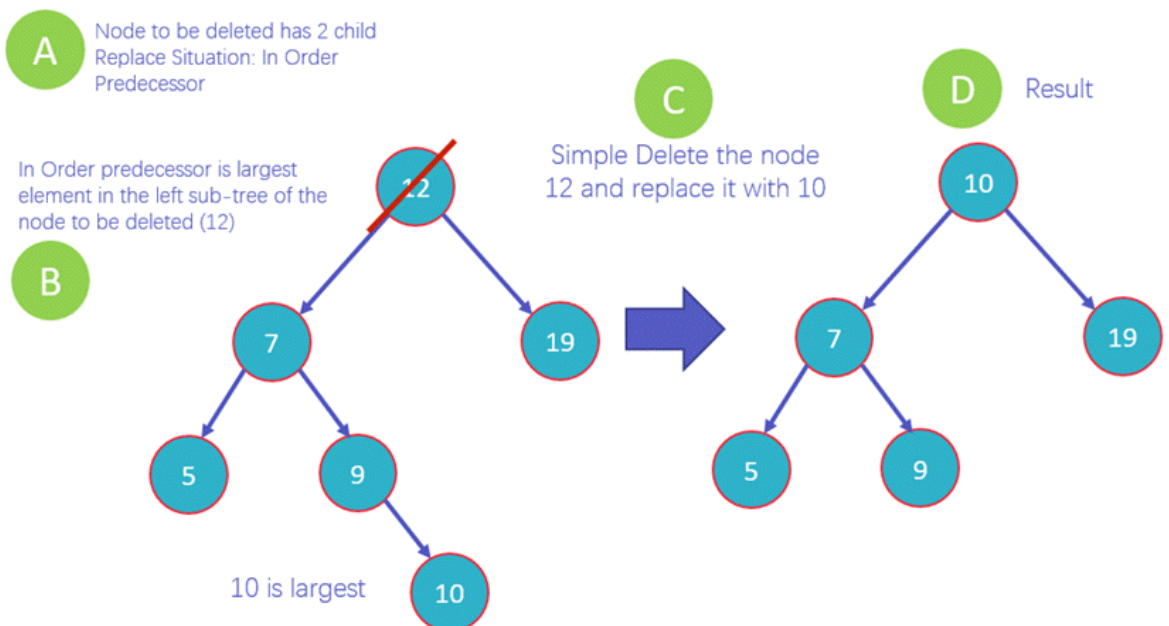
- A. This is the first case of deletion in which you delete a node that has no children. As you can see in the diagram that 19, 10 and 5 have no children. But we will delete 19.
- B. Delete the value 19 and remove the link from the node.
- C. View the new structure of the BST without 19

Delete Operation – Case 2



- This is the second case of deletion in which you delete a node that has 1 child, as you can see in the diagram that 9 has one child.
- Delete the node 9 and replace it with its child 10 and add a link from 7 to 10
- View the new structure of the BST without 9

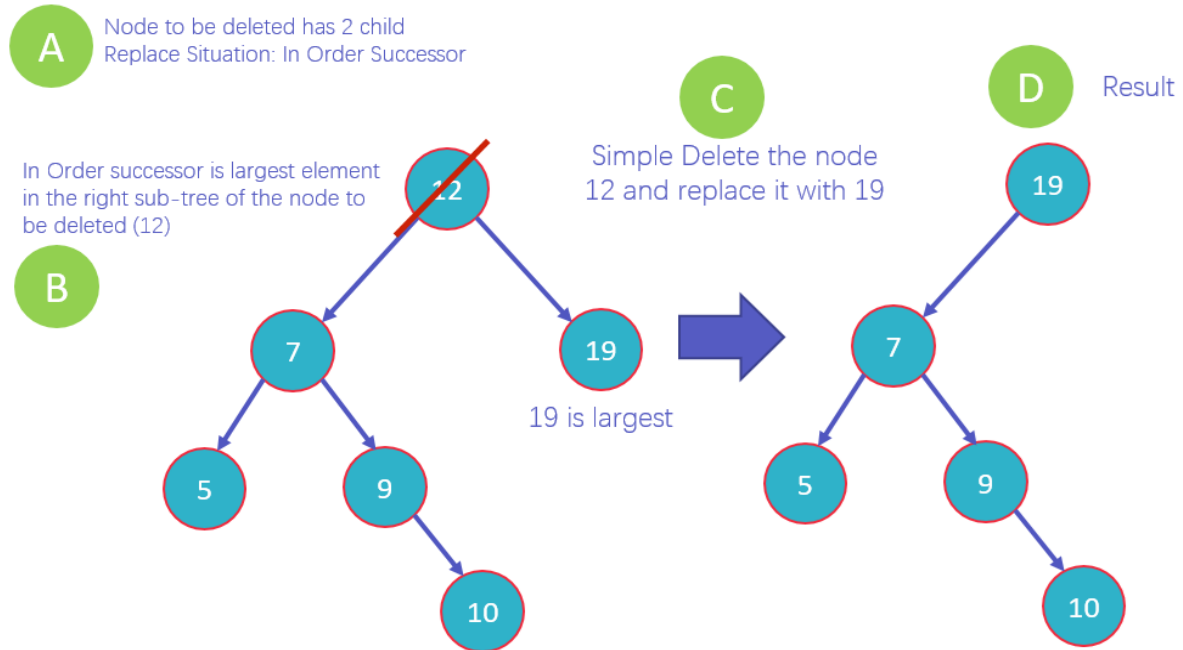
Delete Operation – Case 3 (a)



- Here you will be deleting the node 12 that has two children
- The deletion of the node will occur based upon the in order predecessor rule, which means that the largest element on the left subtree of 12 will replace it.

- C. Delete the node 12 and replace it with 10 as it is the largest value on the left subtree
- D. View the new structure of the BST after deleting 12

Delete Operation – Case 3 (b)



- A. 1 Delete a node 12 that has two children
- B. 2 The deletion of the node will occur based upon the In Order Successor rule, which means that the largest element on the right subtree of 12 will replace it
- C. 3 Delete the node 12 and replace it with 19 as it is the largest value on the right subtree
- D. 4 View the new structure of the BST after deleting 12

Pseudo Code for Deleting a Node:

```

delete (value, root):
    Node x = root
    Node y = NULL
    # searching the node
    while x:
        y = x
        if x.value < value
            x = x.right
        else if x.value > value
            x = x.left
        else if value == x
            break
    # if the node is not null, then replace it with successor
    if y.left or y.right:
        newNode = GetInOrderSuccessor(y)
        root.value = newNode.value
    # After copying the value of successor to the root #we're deleting the successor
    free(newNode)
  
```

```
else  
    free(y)
```

2.5 Summary of Study Unit 2

In this unit, you have learned the following;

- i. Search is a utility that enables its user to search for documents, files, and other types of data. A binary search is an advanced type of search algorithm that finds and fetches data from a sorted list of items.
- ii. Binary search is commonly known as a half-interval search or a logarithmic search
- iii. It works by dividing the array into half on every iteration under the required element is found.
- iv. The binary algorithm takes the middle of the array by dividing the sum of the left and rightmost index values by 2. Now, the algorithm drops either the lower or upper bound of elements from the middle of the array, depending on the element to be found.
- v. The algorithm randomly accesses the data to find the required element. This makes the search cycles shorter and more accurate.
- vi. Binary search performs comparisons of the sorted data based on an ordering principle than using equality comparisons that are slow and inaccurate.
- vii. A binary search is not suitable for unsorted data
- viii. BST is an advanced level algorithm that performs various operations based on the comparison of node values with the root node.
- ix. Any of the points in a parent-child hierarchy represents the node. At least one parent or root node remains present all the time.
- x. There are a left subtree and right subtree. The left subtree contains values that are less than the root node. However, the right subtree contains a value that is greater than the root node.
- xi. Each node can have zero, one, or two children.

- xii. A binary search tree facilitates primary operations like search, insert, and delete.
- xiii. Delete being the most complex have multiple cases, for instance, a node with no child, node with one child, and node with two children.
- xiv. The algorithm is utilized in real-world solutions like games, autocomplete data, and graphics

2.6 Self-Assessment Questions (SAQs) for Study Unit 2

Now that you have completed this study session, you can check to see how well you have achieved its Learning Outcomes by answering the following questions.

SAQ 2.1 (Tests learning outcome 2.1)

- 1. What is do you understand by the term "Binary search"?
- 2. Which type of search is associated with the following concept "Begin at the Beginning"?

SAQ 2.2 (Tests learning outcome 2.2)

- 1. Identify the error in the the following sequential search algorithm and re-write accordingly.

Search (A, t)

- 6. for I = 0 to n -1 do
- 7. If (A [I] = t) then
- 8. return false
- 9. else return true
- 10. End

SAQ 2.3 (Tests learning outcome 2.3)

- 1. The binary search algorithm makes use of mid points for computation. TRUE or FALSE?.

SAQ 2.4 (Tests learning outcome 2.4)

1. Every sub-tree in a binary search tree, is not allowed to have sub-branches on the right and left of themselves (True or False)?
2. Three major types of a binary search tree are....., and

References for further reading

Eiselt, Horst A et al. Integer Programming and Network Models. Springer, 2011.

Introduction to Algorithms (Cormen, Leiserson, Rivest, and Stein) 2001, Chapter 16 "Greedy Algorithms".

Eiselt, Horst A et al. Integer Programming and Network Models. Springer, 2011.

J.H. Holland (1975) *Adaptation in Natural and Artificial Systems*, University of Michigan Press, Ann Arbor, Michigan; re-issued by MIT Press (1992).

Optimal design of heat exchanger networks, Editor(s): Wilfried Roetzel, Xing Luo, Dezhen Chen, Design and Operation of Heat Exchangers and their Networks, Academic Press, 2020, Pages 231-317, ISBN 9780128178942, <https://doi.org/10.1016/B978-0-12-817894-2.00006-6>.

Wang FS., Chen LH. (2013) Genetic Algorithms. In: Dubitzky W., Wolkenhauer O., Cho KH., Yokota H. (eds) *Encyclopedia of Systems Biology*. Springer, New York, NY. https://doi.org/10.1007/978-1-4419-9863-7_412

Fred Glover (1986). "Future Paths for Integer Programming and Links to Artificial Intelligence". *Computers and Operations Research*. **13** (5): 533–549, [https://doi.org/10.1016/0305-0548\(86\)90048-1](https://doi.org/10.1016/0305-0548(86)90048-1)

Optimization of Preventive Maintenance Program for Imaging Equipment in Hospitals, Editor(s): Zdravko Kravanja, Miloš Bogataj, Computer-Aided Chemical Engineering, Elsevier, Volume 38, 2016, Pages 1833-1838, ISSN 1570-7946, ISBN 9780444634283, <https://doi.org/10.1016/B978-0-444-63428-3.50310-6>.

Glover, Fred, and Gary A Kochenberger. *Handbook Of Metaheuristics*. Kluwer Academic Publishers, 2003.

Kirkpatrick, S., Gelatt, C., & Vecchi, M. (1983). Optimization by Simulated Annealing. *Science*, 220(4598), 671-680. Retrieved November 25, 2020, from <http://www.jstor.org/stable/1690046>

Brief review of static optimization methods, Editor(s): Stanisław Sieniutycz, Jacek Jeżowski, Energy Optimization in Process Systems and Fuel Cells (Third Edition), Elsevier, 2018, Pages 1-41, ISBN 9780081025574, <https://doi.org/10.1016/B978-0-08-102557-4.00001-3>.

NIU, M., WAN, C. & Xu, Z. A review on applications of heuristic optimization algorithms for optimal power flow in modern power systems. J. Mod. Power Syst. Clean Energy 2, 289–297 (2014), <https://doi.org/10.1007/s40565-014-0089-4>

J. L. Wang, Y. H. Lin and M. D. Lin, "Application of heuristic algorithms on groundwater pumping source identification problems," 2015 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM), Singapore, 2015, pp. 858-862, <https://doi.org/10.1109/IEEM.2015.7385770>.

Matott, L. Shawn, et al. "Application of Heuristic Optimization Techniques and Algorithm Tuning to Multilayered Sorptive Barrier Design." Environmental Science & Technology, vol. 40, no. 20, 2006, pp. 6354–6360., <https://doi.org/10.1021/es052560+>.

Larranaga P, Calvo B, Santana R, Bielza C, Galdiano J, Inza I, Lozano JA, Armananzas R, Santafe G, Perez A, Robles V (2006) Machine learning in bioinformatics. Brief Bioinform 7(1):86–112

Study Unit 3: Sorting Algorithms

Expected Duration: # week

Introduction

In Computer Programming, there are situations that require a rearrangement of records or items in a data base for easier access. This process necessitates a concept known as Sorting. In this unit, we shall be discussing the essence of sorting algorithms. We shall also be looking at sorting algorithms especially Insertion sort and Merge sort algorithms.

[Learning Outcomes for Study Unit 3](#)

At the end of this study unit, you should be able to;

- 3.1 Define and use correctly the words in bold (SAQ 3.1)
- 3.2. Write and explain the Insertion Sort algorithm (SAQ 3.2)
- 3.3. Write and explain the Merge Sort algorithm (SAQ 3.3)

3.1 The Concept of Sorting and Sorting Algorithm

Sorting is a topic that arises very frequently in programming: The rearrangement of items into ascending or descending order. Quite remarkably, the order in which items are stored in computer memory often has a profound influence on the speed and simplicity of algorithms that manipulate those items. In contrast to other terminologies of similar meaning, such as ordering or sequencing, sorting has become firmly established in computing parlance in the strict sense of sorting into order.

3.1.1 Some of the most important applications of sorting are;

- (i) Grouping items of identical values together: this can be achieved by arranging items in ascending or descending order.
- (ii) Matching items in two or more files: If several files have been sorted into the same order, it is possible to find all the matching entries in one sequential pass through them, without backing up.

- (iii) Searching for information by key values: sorting is an important aid to searching. It helps, to make computer output, and indeed most texts, more suitable for human consumption. Simply imagine how labyrinthine it can be to search for a word in the dictionary without sorting, etc.

3.1.2 The concept of Sorting

Before proceeding to design an example sorting algorithm, we define our sorting problem more clearly and look at some terminology (Knuth, 1998). Consider N items. $R_1, R_2, \dots, R_N$ to be sorted; each R^{th} item is a record, and the entire collection of N records is called a file. Each record R_j has a key K_j , which governs the sorting process. Additional data, besides the key, is usually also present; this entire information has no effect on sorting except that it must be carried along as part of each record.

An ordering relation “ $<$ ” is specified on the keys so that the following conditions are satisfied for any key values a, b, c :

- (i) Exactly one of the possibilities; $a < b$, $a = b$, or $b < a$ is true (the trichotomy rule).
- (ii) If $a < b$, and $b < c$, then $a < c$ (law of transitivity)

Notably, properties (i) and (ii) characterize the mathematical concept of linear ordering, also called total ordering. Any relationship “ $<$ ” satisfying these two properties can be sorted by most methods, although some sorting techniques are designed to work only with numerical or alphabetic keys that have the usual ordering.

It is also important to mention that sorting can be classified generally into internal and external sorting. With internal sorting, the records are kept entirely in the computer’s high-speed random-access memory, which allows for more flexibility in the structuring and accessing of the data. While external sorting becomes necessary when there are more records than can be stored in the RAM, implying the need for external storage, but poses some access constraints.

3.2 Insertion sort:

We shall be using the following illustration to explain the concept of Insertion sort:

- a. Insertion sort works the way many people sort a hand of playing cards.
- b. Usually, people start with an empty left hand and the cards face down on the table.
- c. Then one card is removed at a time from the table and inserted into the correct position in the left hand.
- d. To find correct position for a card, we compare it with each of the cards already in the hand, from right to left.
- e. At all times, the cards held in the left hand are sorted, and these cards, were originally the top cards of the pile on the table.
- f. We therefore, proceed to present our sorting algorithm, called INSERTION SORT.

Recall our already introduced sorting problem where;

Input: A Sequence of numbers $\langle a_1, a_2, \dots, a_n \rangle$

Output: A permutation (recording) $\langle a_1, a_2, \dots, a_n \rangle$ of the input sequence $\ni a_1^1 \leq a_2^1 \leq \dots \leq a_n^1$

The numbers to be sorted are known as the keys. Although conceptually, we are sorting a sequence, the input comes to us in the form of an array with n – elements. Thus, insertion sort takes as parameter an array $A[1..n]$ containing a sequence of length n that is to be sorted (i.e cards on table).

The algorithm sorts the input numbers in place, which implies that it rearranges the numbers within the Array A , with at most a constant number of them stored outside the array at any time.

The input array A contains the sorted output sequence when the INSERTION-SORT procedure is finished.

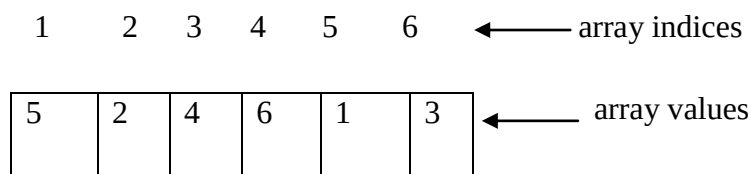
3.2.1 Insertion Sort Algorithm

```
1   For   j   = 2 to A.length //it's still ok to say A.n
2       key = A [ j ]
3       // insert A[j] into the sorted sequence A[1 ..j -1]
4       i   = j - 1
5       while i > 0 and A [i] > key /*while loop does not stop until A[i]
```

< key or i = 0 */

6. $A[i+1] = A[i]$ // rightward shift (Interim insertion)
7. $i = i - 1$
8. $A[i + 1] = \text{key}$ //insert then transfer control to the for loop for next j

To demonstrate the effectiveness of the INSERTION SORT algorithm, consider the array below;



Loop Invariants and Correctness of Insertion Sort

First of all, we note that a loop invariant is a condition that is necessarily true immediately before and immediately after each iteration of a loop.

Considering the given algorithm in section 3.2.1;

- a. The index j indicates the current card being inserted into the hand.
- b. At the beginning of each interaction of the “for loop”, which is indexed by j, the subarray consisting of elements $A[1 \dots j - 1]$ constitutes the currently sorted array (hand), and the remaining subarray $A[j + 1 \dots n]$ corresponds to the pile of cards still on the table.
- c. In fact, elements $A[1 \dots j - 1]$ are the elements originally in positions 1 through j – 1, but now in sorted order (this is the responsibility of the loop indexed by i : imagine an interim subarray)
- d. These properties of $A[1 \dots j - 1]$ is formally stated as a loop invariant (Cormen, et al, 2009).
At the start of each interaction of the “for” loop of lines 1 – 8, the subarray $A[1 \dots j - 1]$ consists of the elements originally in $A[1 \dots j - 1]$, but in sorted order.
- e. Loop invariants can be used to understand why an algorithm is correct by considering three properties, initialization, maintenance, and termination (Cormen, et al, 2009).

Where;

- Initialization - is time prior to the first iteration of the loop.
- Maintenance - is true before an iteration of the loop, it remains true before the next iteration.

f. Termination - When the loop terminates, the invariant gives a useful property that helps to show that the algorithm is correct.

Considering the example array of values given;

5	2	4	6	1	3
---	---	---	---	---	---

1 2 3 4 5 6

g. In the context of the loop invariants; for initialization:

$J = 2$ by the first iteration.

- The second property, maintenance, provides that each of the iterations maintains the loop invariant. Informally, the body of the ‘for’ loop works by moving $A[j-1]$, $A[j-2]$, $A[j-3]$, and so on by one position to the right until it finds the proper position for $A[j]$ (lines 4 – 7), at which point it inserts the value of $A[j]$ (line 8). The subarray $A[1 \dots j-1]$ then consists of the elements originally in $A[1 \dots j-1]$, but in sorted order. Incrementing j for the next iteration of the ‘for’ loop then preserves the loop invariant.
- Termination - the loop terminates when for the “for loop”, $j > A.length = n$; at a point where $j = n + 1$; as each loop iteration increases j by 1 and the sorted array becomes $A[1 \dots n]$ in sorted order.

In-text Question: One real life application of sorting is?

Solution: Matching items in two or more files

3.3 The Merge Sort Algorithm

This is another type of sorting algorithm. The design approach used for the insertion sort is described as an incremental approach, where a single element $A[j]$ was inserted into its proper place, yielding the sorted subarray $A[1 \dots j - 1]$.

An alternative design approach known as the divide-and-conquer is used to design a sorting algorithm whose worst case running time can become much less than that of insertion sort (more efficient).

The algorithms that follow the divide-and-conquer approach are those that break a problem into several sub problems that are similar to the original problem but smaller in size, solve the sub problems recursively, and then combine these solutions to create a solution to the original problem.

The merge sort algorithm follows the divide-and-conquer approach. The algorithm operates in the following manner;

- (i) Divide the n – element sequence to be sorted into two subsequences of $n/2$ elements each.
- (ii) Conquer by sorting the two subsequences recursively using merge sort.
- (iii) Combine by merging the two sorted sequences to produce the sorted answer.

The key operation of the merge sort algorithm is the merging of two sorted sequences in the “combine” step.

The merge operation becomes effective by calling an auxiliary procedure; MERGE (A , P , q , r)

Where; A is an array and p , q , and r are indices into the array such that.

$$P \leq q < r.$$

- The procedure assumes that the subarrays $A[P \dots q]$ and $A[q + 1 \dots r]$ are in sorted order..
- The procedure therefore merges them to form a single sorted subarray that replaces the current subarray $A[P \dots r]$

- We shall come to see in the section on efficiency that the procedure MERGE takes time $\Theta(n)$, where $n = r - p + 1$ is the total number of elements being sorted.
- To understand how MERGE SORT works, consider our card sorting example, but this time suppose we have two piles of cards face up on a table
- Each card is sorted with the smallest cards on top.
- We wish to merge the two piles into a single sorted output pile, which is to be faced down on the table.
- The basic step is to choose the smaller of the two cards on top of the face-up-piles, removing it from; its pile (which exposes a new top card), and placing this card face down into the output pile.
- Computationally, each basic step takes a constant time, albeit a simplifying assumption, since we are comparing just the two top cards. As we perform at most “ n ” basic steps, merging takes time $\Theta(n)$.
- The following pseudocode implements the above idea, but with an additional twist that avoids having to check whether either pile is empty in each basic step.
- We place at the bottom of each file a sentinel card (i.e. a guard), denoted by ∞ , for reasons of simplicity.
- Once both sentinel cards are exposed, all the nonsentinel cards have already been placed on to the output pile.
- Since we know in advance that exactly $r - p + 1$ number of cards will be placed on to the output file, we can stop once we have performed that many basic steps.

3.3.1 Merge Sort Algorithm

As a necessary step, we first of all describe a utility which the merge sort algorithm calls to combine two already sequences/arrays into a single sorted array. That utility, another program, is called the Merge procedure with the following description:

MERGE (A, P, q, r)

1. $n_1 = q - P + 1$

2. $n_2 = r - q$
3. Let $L[1 \dots n_1+1]$ and $R[1 \dots n_2+1]$ //be new arrays each with space for the sentinels.
4. for $i = 1$ to n_1
5. $L[i] = A[P + i-1]$
6. for $j = 1$ to n_2
7. $R[j] = A[q + j]$
8. $L[n_1 + 1] = \infty$
9. $R[n_2 + 1] = \infty$
10. $i = 1$
11. $j = 1$
12. for $k = P$ to r
13. if $L[i] \leq R[j]$
14. $A[k] = L[i]$
15. $i = i + 1$
16. else $A[k] = R[j]$
17. $j = j + 1$

Description (in detail)

- a. Line 1 computes the length n_1 of the subarray $A[P \dots q]$
- b. Line 2 computes the length n_2 of the subarray $A[q + 1 \dots r]$
- c. Line 3 creates the arrays L and R of lengths $n_1 + 1$ and $n_2 + 1$ respectively to accommodate the sentinels.
- d. Lines 4 – 5: the “for” loop copies subarray $A[P \dots q]$ into $L[1 \dots n_1]$
- e. Lines 6 – 7: the “for” Loop copies the subarray $A[q + 1 \dots r]$ into $R[1 \dots n_2]$
- f. Lines 8 – 9: put the sentinels at the ends of the arrays L and R .
- g. Lines 10 – 17: perform the $r - p + 1$ basic steps by maintaining the loop invariant.

In the MERGE procedure, lines 10 to 17 perform the $r - p + 1$ basic steps by maintaining the following loop invariant:

At the start of each iteration of the “for” loop of lines 12 to 17, the subarray $A[p \dots k-1]$ contains the $k - p$ smallest elements of $L[1 \dots n_1+1]$ and $R[1 \dots n_2 + 1]$, in sorted order.

Moreover, $L[i]$ and $R[j]$ are the smallest elements of their array that have not been copied back to A .

It is pertinent to state that the merge procedure is the combine part of the merge sort algorithm. In reality, the MERGE procedure is a subroutine in MERGE SORT algorithm. The procedure MERGE-SORT(A, p, r) sorts the elements in the subarray $A[p \dots r]$. If $p \geq r$, the subarray has at most one element and is therefore already sorted. Otherwise, the divide step simply computes an index q that partitions $A[p \dots r]$ into two subarrays; $A[p \dots q]$ containing $\lceil n/2 \rceil$ elements and $A[q+1 \dots r]$ containing $\lfloor n/2 \rfloor$ elements also. Thus, and in essence, the merge sort algorithm is a typical example of divide-and-conquer algorithm, exhibiting the well known characteristic of divide, conquer, and combine steps of such algorithms.

In-text Question: One real life application of sorting is?

Solution: Matching items in two or more files

3.3.2 The Merge Sort Algorithm Generalised

We therefore provide a description of the Merge Sort algorithm:

MERGE-SORT (A, p, r)

1. if $p < r$
2. $q = \lfloor (p + r)/2 \rfloor$ //using the floor/ceiling function for odd numbered arrays
3. MERGE-SORT (A, p, q)
4. MERGE-SORT ($A, q+1, r$)
5. MERGE (A, p, q, r)
6. Return $A[p \dots r]$

To sort the entire sequence $A\{A[1], A[2], \dots, A[n]\}$, we make the initial call MERGE-SORT ($A, 1, A.length$), where $A.length = n$. This can be illustrated using the operation of the procedure bottom-up, where n is power of 2. The algorithm consists of merging pairs of 1-item sequences to form sorted sequences of length 2 (ie 2-item sorted sequences), merging pairs of sequences of length 2 to form sequences of length 4, and so on until two sequences of length $n/2$ are merged to form the final sorted sequence of length n .

3.4 Summary of Study Unit 3

In this unit, you have learned the following;

- i. Sorting means the rearrangement of items in into ascending or descending order.
- ii. Sorting can be classified as internal or external sorting.
- iii. Internal sorting requires that the records are kept entirely in the computer's high-speed random-access memory, while external makes use of external storage.
- iv. Insertion sort takes as parameter an array $A[1...n]$ containing a sequence of length n that is to be sorted.
- v. The Insertion sort algorithm sorts the input numbers in place, which implies that it rearranges the numbers within the Array A , with at most a constant number of them stored outside the array at any time.
- vi. The merge sort algorithm follows the divide-and-conquer approach.
- vii. The merge sort algorithm breaks a problem into several sub problems that are similar to the original problem but smaller in size, and proceeds to solve the sub problems recursively, and then combine these solutions to create a solution to the original problem.

3.5 Self-Assessment Questions (SAQs) for Study Unit 3

Now that you have completed this study session, you can check to see how well you have achieved its Learning Outcomes by answering the following questions.

SAQ 3.1 (Tests learning outcome 3.1)

1. The rearrangement of items in into ascending or descending order in computing is known as?
2. Sorting can search for items by making use of by key values (TRUE or FALSE)?

SAQ 3.2 (Tests learning outcome 3.2)

1. Given the following:

5	2	4	6	1	3
---	---	---	---	---	---

If index key is given as $[0, \dots, n]$, What will be the index of the array content 4?

SAQ 3.3 (Tests learning outcome 3.3)

1. The term "divide-and-conquer" is associated to what type of sorting technique?

References for further reading

Eiselt, Horst A et al. Integer Programming and Network Models. Springer, 2011.

Introduction to Algorithms (Cormen, Leiserson, Rivest, and Stein) 2001, Chapter 16 "Greedy Algorithms".

Eiselt, Horst A et al. Integer Programming and Network Models. Springer, 2011.

J.H. Holland (1975) *Adaptation in Natural and Artificial Systems*, University of Michigan Press, Ann Arbor, Michigan; re-issued by MIT Press (1992).

Optimal design of heat exchanger networks, Editor(s): Wilfried Roetzel, Xing Luo, Dezhen Chen, Design and Operation of Heat Exchangers and their Networks, Academic Press, 2020, Pages 231-317, ISBN 9780128178942, <https://doi.org/10.1016/B978-0-12-817894-2.00006-6>.

Wang FS., Chen LH. (2013) Genetic Algorithms. In: Dubitzky W., Wolkenhauer O., Cho KH., Yokota H. (eds) *Encyclopedia of Systems Biology*. Springer, New York, NY. https://doi.org/10.1007/978-1-4419-9863-7_412

Fred Glover (1986). "Future Paths for Integer Programming and Links to Artificial Intelligence". *Computers and Operations Research*. **13** (5): 533–549, [https://doi.org/10.1016/0305-0548\(86\)90048-1](https://doi.org/10.1016/0305-0548(86)90048-1)

Optimization of Preventive Maintenance Program for Imaging Equipment in Hospitals, Editor(s): Zdravko Kravanja, Miloš Bogataj, Computer-Aided Chemical Engineering, Elsevier, Volume 38, 2016, Pages 1833-1838, ISSN 1570-7946, ISBN 9780444634283, <https://doi.org/10.1016/B978-0-444-63428-3.50310-6>.

Glover, Fred, and Gary A Kochenberger. *Handbook Of Metaheuristics*. Kluwer Academic Publishers, 2003.

Kirkpatrick, S., Gelatt, C., & Vecchi, M. (1983). Optimization by Simulated Annealing. *Science*, 220(4598), 671-680. Retrieved November 25, 2020, from <http://www.jstor.org/stable/1690046>

Brief review of static optimization methods, Editor(s): Stanisław Sieniutycz, Jacek Jeżowski, Energy Optimization in Process Systems and Fuel Cells (Third Edition), Elsevier, 2018, Pages 1-41, ISBN 9780081025574, <https://doi.org/10.1016/B978-0-08-102557-4.00001-3>.

NIU, M., WAN, C. & Xu, Z. A review on applications of heuristic optimization algorithms for optimal power flow in modern power systems. J. Mod. Power Syst. Clean Energy 2, 289–297 (2014), <https://doi.org/10.1007/s40565-014-0089-4>

J. L. Wang, Y. H. Lin and M. D. Lin, "Application of heuristic algorithms on groundwater pumping source identification problems," 2015 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM), Singapore, 2015, pp. 858-862, <https://doi.org/10.1109/IEEM.2015.7385770>.

Matott, L. Shawn, et al. "Application of Heuristic Optimization Techniques and Algorithm Tuning to Multilayered Sorptive Barrier Design." Environmental Science & Technology, vol. 40, no. 20, 2006, pp. 6354–6360., <https://doi.org/10.1021/es052560+>.

Larranaga P, Calvo B, Santana R, Bielza C, Galdiano J, Inza I, Lozano JA, Armananzas R, Santafe G, Perez A, Robles V (2006) Machine learning in bioinformatics. Brief Bioinform 7(1):86–112

Study Unit 4: Hash Tables, Graphs and Representation of Graphs

Expected Duration: # week

Introduction

This unit presents two other important data structures, which are often regarded as dynamic data sets in data structures and algorithms. These dynamic data structures are data structures that can grow or shrink as they are required to allocate and de-allocate memory spaces at run time. The organisation of this unit attempts to satisfy the content requirements of this course and not necessarily to depict relatedness of the topics presented here. Thus, the topics covered here include those of hash tables, and then graphs and representation of graphs.

Learning Outcomes for Study Unit 4

At the end of this study unit, you should be able to;

- 4.1 Define and use correctly the words in bold (SAQ4.1)
- 4.2. List the components of hashing and associated methods (SAQ 4.2)
- 4.3. List and explain the types of graph algorithms and applications (SAQ 4.3)

4.1 What is Hashing?

Hashing is a popular technique in computer science that involves mapping large data sets to fixed-length values. It is a process of converting a data set of variable size into a data set of a fixed size. The ability to perform efficient lookup operations makes hashing an essential concept in data structures

4.1.1 Hashing Algorithm

A hashing algorithm is used to convert an input (such as a string or integer) into a fixed-size output (referred to as a hash code or hash value). The data is then stored and retrieved using this hash value as an index in an array or hash table. The hash function must be deterministic, which guarantees that it will always yield the same result for a given input.

Hashing is commonly used to create a unique identifier for a piece of data, which can be used to quickly look up that data in a large dataset. For example, a web browser may use hashing to store website passwords securely. When a user enters their password, the browser converts it into a hash value and compares it to the stored hash value to authenticate the user.

There are numerous hashing algorithms, each with distinct advantages and disadvantages. For the purpose of this module, we shall just mention them here as follows:

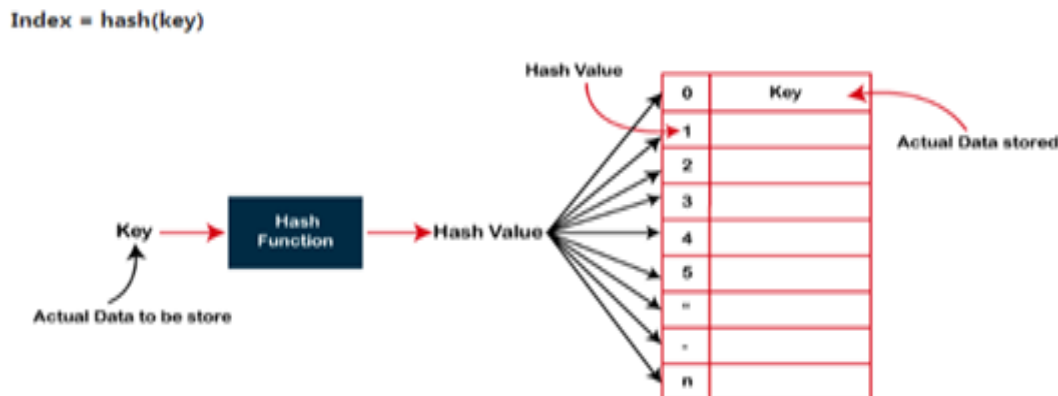
- i. MD5: A widely used hashing algorithm that produces a 128-bit hash value.
- ii. SHA-1: A popular hashing algorithm that produces a 160-bit hash value.
- iii. SHA-256: A more secure hashing algorithm that produces a 256-bit hash value.

4.1.2 Components of Hashing

The major components in a hashing algorithm are:

- a. **Key:** A key is any text or number that is provided as input into a hash function, a method for determining an item's index or placement in a data structure.
- b. **Hash Function:** The hash function takes a key as input and outputs the index of a hash table's entry. The index is sometimes referred to as a hash index.
- c. **Hash Table:** A hash table is a type of data structure that uses a unique function called a hash function to map keys to values. Hash holds the data in an array in an associative fashion, giving each data value a distinct index.

The process of hashing could be summarised in the following diagram



4.1.3 Methods of implementing a Hash Function

There are different ways of calculating the hash function:

- i. Division method:- This method involves dividing the key by the table size and taking the remainder as the hash value. For example, if the table size is 10 and the key is 23, the hash value would be 3 ($23 \% 10 = 3$).
- ii. Multiplication method:- This method involves multiplying the key by a constant and taking the fractional part of the product as the hash value. For example, if the key is 23 and the constant is 0.618, the hash value would be 2 ($\text{floor}(10 * (0.61823 - \text{floor}(0.61823))) = \text{floor}(2.236) = 2$).
- iii. Folding method

- iv. Mid square method

4.1.4 Collision in Hashing

When the two different values have the same value, then the problem occurs between the two values, known as a collision. In the above example, the value is stored at index 6. If the key value is 26, then the index would be:

$$h(26) = 26 \% 10 = 6$$

Therefore, two values are stored at the same index, i.e., 6, and this leads to the collision problem. To resolve these collisions, we have some techniques known as collision techniques.

In-text Question: Can a hash key be a string?

Solution: Yes

There are two approaches to solving the problem of collision in hashing. They are

1. Separate Chaining or Open Hashing

The goal is to create a linked list of records with the same hash function value that each cell of the hash table may point to. Although chaining is straightforward, extra memory outside the table is needed.

Example: Using a different chaining mechanism for collision resolution, we must enter certain entries into the hash table after receiving the hash function.

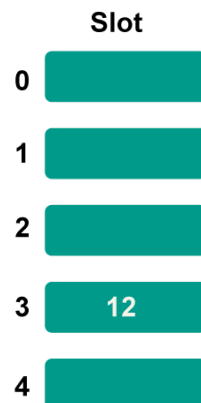
1. Hash function = $\text{key} \% 5$,
2. Elements = 12, 15, 22, 25 and 37.

Let's look at a step-by-step solution to the aforementioned issue:

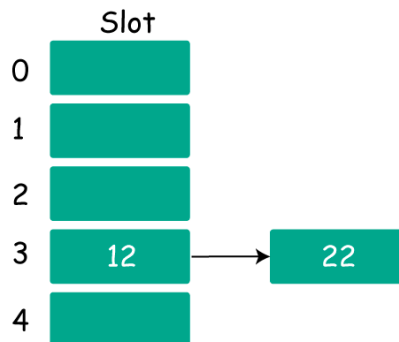
Step 1: Create an empty hash table with a potential range of hash values between 0 and 4, using the specified hash algorithm.

Slot	
0	
1	
2	
3	
4	

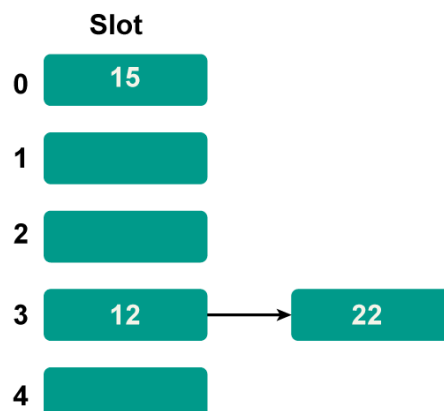
Step 2: Now add each key individually to the hash table. The hash function $12\%5=2$ is used to determine the bucket number 2 for the first key to be added, which are 12.



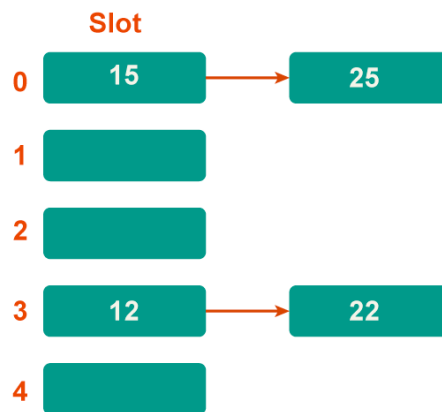
Step3: Now 22 is the current next key. Since $22\%5=2$, it will correspond to bucket number 2. Key 12 has already occupied bucket 2 however.



Step 4: The next key is 15. Since $15\%5=0$, it will map to slot number 0.



Step 5: Now 25 is the current next key. It will have a bucket number of $25\%5=0$. Key 25 has already occupied bucket 0 however. In order to address the collision, the separate chaining technique will once more create a linked list with bucket 0.



Hence In this case, the collision resolution approach is the separate chaining method.

2 Open Addressing or Closed Hashing

In open addressing, the hash table alone houses all of the components. Either a record or NIL is present in every table entry. When looking for an element, we go through each table slot individually until the sought-after element is discovered or it becomes obvious that the element is not in the table.

a) Linear Probing

In linear probing, the hash table is systematically examined beginning at the hash's initial point. If the site we get is already occupied, we look for a different one.

Algorithm:

- i. Calculate the hash key. i.e. $\text{key} = \text{data} \% \text{size}$
- ii. Check, if $\text{hashTable}[\text{key}]$ is empty store the value directly by $\text{hashTable}[\text{key}] = \text{data}$
- iii. If the hash index already has some value then check for next index using $\text{key} = (\text{key} + 1) \% \text{size}$
- iv. Check, if the next index is available $\text{hashTable}[\text{key}]$ then store the value. Otherwise try for next index.
- v. Do the above process till we find the space.

An Example: Consider the following basic hash function, "key mod 5", with the following keys to be inserted: 50, 70, 76, 85, and 93.

Step 1: First, create an empty hash table with a potential range of hash values between 0 and 4, using the specified hash algorithm.



Step 2: Now add each key individually to the hash table. 50 is the first key. Because $50\%5=0$, it will map to slot number 0. Therefore, place it in slot number 0.



Step 3: Now 70 is the next key. Since 50 is already in slot number 0, it will map to slot number 0 since $70\%5=0$, therefore look for the next vacant slot and place it there.



Step 4: Now 76 is the next key. Because $76\%5=1$, it will map to slot number 1, but because slot number 1 is already occupied by 70, find the following vacant slot and insert it.

Slot	
0	50
1	70
2	76
3	
4	

Step 5: Now 93 is the next key. Because $93\%5=3$, it will map to slot number 3, so put it there.

Slot	
0	50
1	70
2	76
3	93
4	

In-text Question: In linear probing, hashing commences at the bottom of the table (TRUE or FALSE)?

Solution: FALSE

b) Quadratic Probing

In computer programming, quadratic probing is an open addressing approach for resolving hash collisions in hash tables. Until an available slot is discovered, quadratic probing works by adding consecutive values of any arbitrary quadratic polynomial to the initial hash index.

4.1.5 Real life Applications of Hashing

Hash data structure applications in real life include:

- Cache mapping uses hash for quick access to the data.
- Password verification may be done using hashes.
- As a message digest in cryptography, hash is utilized.

- d. For matching patterns in a string, use the Rabin-Karp method.
- e. Figuring out how many distinct substrings a string has.

4.2 Graph Algorithms

The graph data structure is used to store data required in computation to solve many computer programming problems. Graphs are used to address real-world problems in which the problem area is represented as a network, such as **telephone networks**, **circuit networks**, **LinkedIn**, **Facebook**, etc.

In computer science, The graph is an abstract data type used to implement the undirected and directed graph notions from graph theory in mathematics.

A graph data structure is made up of a finite and potentially mutable set of vertices (also known as nodes or points), as well as a set of unordered pairs for an undirected graph or a set of ordered pairs for a directed graph. These pairs are recognized as edges, links, or lines in a directed graph but are also known as arrows or arcs. The vertices could be internal graph elements or external items represented by integer indices or references.

4.2.1 Types of Graphs

So depending upon the position of these nodes and vertices, there are different types of graphs, such as:

- i. Null Graph
- ii. Trivial Graph
- iii. Non-directed Graph
- iv. Directed Graph
- v. Connected Graph
- vi. Disconnected Graph
- vii. Regular Graph
- viii. Complete Graph
- ix. Cycle Graph
- x. Cyclic Graph
- xi. Acyclic Graph
- xii. Finite Graph
- xiii. Infinite Graph
- xiv. Bipartite Graph
- xv. Planar Graph
- xvi. Simple Graph

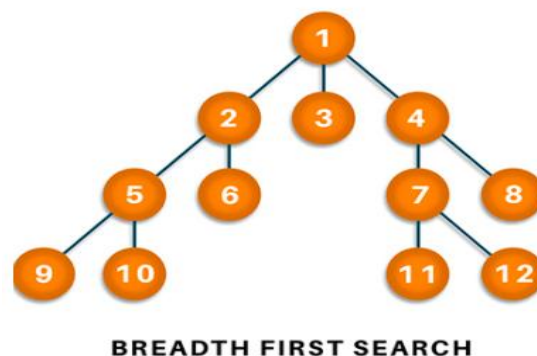
- xvii. Multi Graph
- xviii. Pseudo Graph
- xix. Euler Graph
- xx. Hamiltonian Graph

4.2.2 Types of Graph Algorithms

There are various types of graph algorithms. In this module we shall consider the following:

1. Breadth-First Search

Traversing or searching is one of the most used operations that are undertaken while working on graphs. Therefore, in **breadth-first-search** (BFS), you start at a particular vertex, and the algorithm tries to visit all the neighbors at the given depth before moving on to the next level of traversal of vertices. Unlike trees, graphs may contain cyclic paths where the first and last vertices are remarkably the same always. Thus, in BFS, you need to keep note of all the track of the vertices you are visiting. To implement such an order, you use a queue data structure which First-in, First-out approach. To understand this, see the image given below.



BFS Algorithm

Breadth First Search Algorithm is given as:

1. Start putting any one vertices from the graph at the back of the queue.
2. First, move the front queue item and add it to the list of the visited node.
3. Next, create nodes of the adjacent vertex of that list and add them which have not been visited yet.
4. Keep repeating steps two and three until the queue is found to be empty.

Breadth First Search Pseudocode

```
1. Set all nodes to "not visited";
2.  q = new Queue();
3.  q.enqueue(initial node);
4.  while ( q ? empty ) do
5.  {
6.      x = q.dequeue();
7.      if ( x has not been visited )
8.      {
9.          visited[x] = true;      // Visit node x !
10.
11.         for ( every edge (x, y) /* we are using all edges ! */ )
12.             if ( y has not been visited )
13.                 q.enqueue(y);    // Use the edge (x,y) !!!
14.     }
15. }
```

Applications of BFS

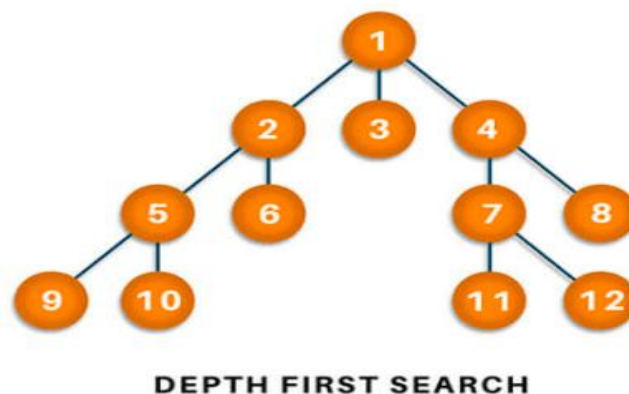
BFS algorithm has various applications. For example, it is used to determine the **shortest path** and **minimum spanning tree**. It is also used in web crawlers to create web page indexes. It is also used as powering search engines on social media networks and helps to find out peer-to-peer networks in BitTorrent.

In-text Question: In BFS algorithm, traversing commencing at the bottom of the tree or leaf. (TRUE or FALSE)

Solution: FALSE

2. Depth-first search

In depth-first-search (DFS), you start by particularly from the vertex and explore as much as you along all the branches before backtracking. In DFS, it is essential to keep note of the tracks of visited nodes, and for this, you use stack data structure.



Depth First Search (DFS) Algorithm

1. Start by putting one of the vertexes of the graph on the stack's top.
2. Put the top item of the stack and add it to the visited vertex list.
3. Create a list of all the adjacent nodes of the vertex and then add those nodes to the unvisited at the top of the stack.
4. Keep repeating steps 2 and 3, and the stack becomes empty.

Depth First Search Pseudocode

1. DFS(G, v) (v is the vertex where the search starts)
2. Stack $S := \{\}$; (start with an empty stack)
3. for each vertex u , set $\text{visited}[u] := \text{false}$;
4. push S, v ;
5. while (S is not empty) do
6. $u := \text{pop } S$;
7. if (not $\text{visited}[u]$) then
8. $\text{visited}[u] := \text{true}$;
9. for each unvisited neighbour w of u
10. push S, w ;
11. end if
12. end while
13. END DFS()

Applications of DFS

DFS finds its application when it comes to finding paths between two vertices and detecting cycles. Also, topological sorting can be done using the DFS algorithm easily. DFS is also used for one-solution puzzles.

4.4 Summary of Study Unit 4

In this unit, you have learned the following;

- i. Hashing is commonly used to create a unique identifier for a piece of data, which can be used to quickly look up that data in a large dataset
- ii. A hashing algorithm is used to convert an input into a fixed-size output.

- iii. The three main components of hashing are; key, hash function and hash table.
- iv. Methods of implementing a Hash Function include; Division method, Multiplication method, folding method and Mid-square method.
- v. When the two different values have the same value, then the problem occurs between the two values, known as a collision.
- vi. The two main techniques for preventing collision are Separate Chain method and Open Addressing method.
- vii. Hashing can be applied in cryptography, password verification, matching patterns and several other areas.
- viii. The graph data structure is used to store data required in computation to solve many computer programming problems
- ix. A graph data structure is made up of a finite and potentially mutable set of vertices (also known as nodes or points), as well as a set of unordered pairs for an undirected graph or a set of ordered pairs for a directed graph.
- x. The two major types of graph algorithms are Breadth first search and Depth first search.
- xi. BFS starts from the vertex or top while DFS starts from the branches or bottom.

4.5 Self-Assessment Questions (SAQs) for Study Unit 4

Now that you have completed this study session, you can check to see how well you have achieved its Learning Outcomes by answering the following questions.

SAQ 4.1 (Tests learning outcome 4.1)

- 1. A popular technique in computer science that involves mapping large data sets to fixed-length values is known as?
- 2. A hashing algorithm converts a string or integer into a fixed-size hash value (TRUE or FALSE)?

SAQ 4.2 (Tests learning outcome 4.2)

1. what component of hashing takes a key as input and outputs the index of a hash table's entry ?
2. Given the following: $(29 \% 10 = 3)$, what method of hash function was applied?
3. When the two different values have the same value, resulting to a between the two values, such situation is known as?
4. Linear probing can be classified under closed or open hashing management for collision?

SAQ 4.3 (Tests learning outcome 4.3)

1. Can graphs be used to address social network problems? (Yes/No)?
2. One of these is NOT a type of graph; Infinite Graph, Bipartite Graph, Planar Graph, hollow graph
3. What type of graph algorithm is best suited for addressing shortest path problems?
4. The term backtracking is associated to what type of graph algorithm?

References for further reading

Eiselt, Horst A et al. Integer Programming and Network Models. Springer, 2011.

Introduction to Algorithms (Cormen, Leiserson, Rivest, and Stein) 2001, Chapter 16 "Greedy Algorithms".

Eiselt, Horst A et al. Integer Programming and Network Models. Springer, 2011.

J.H. Holland (1975) *Adaptation in Natural and Artificial Systems*, University of Michigan Press, Ann Arbor, Michigan; re-issued by MIT Press (1992).

Optimal design of heat exchanger networks, Editor(s): Wilfried Roetzel, Xing Luo, Dezhen Chen, Design and Operation of Heat Exchangers and their Networks, Academic Press, 2020, Pages 231-317, ISBN 9780128178942, <https://doi.org/10.1016/B978-0-12-817894-2.00006-6>.

Wang FS., Chen LH. (2013) Genetic Algorithms. In: Dubitzky W., Wolkenhauer O., Cho KH., Yokota H. (eds) *Encyclopedia of Systems Biology*. Springer, New York, NY. https://doi.org/10.1007/978-1-4419-9863-7_412

Fred Glover (1986). "Future Paths for Integer Programming and Links to Artificial Intelligence". *Computers and Operations Research*. **13** (5): 533–549, [https://doi.org/10.1016/0305-0548\(86\)90048-1](https://doi.org/10.1016/0305-0548(86)90048-1)

Optimization of Preventive Maintenance Program for Imaging Equipment in Hospitals, Editor(s): Zdravko Kravanja, Miloš Bogataj, Computer-Aided Chemical Engineering, Elsevier, Volume 38, 2016, Pages 1833-1838, ISSN 1570-7946, ISBN 9780444634283, <https://doi.org/10.1016/B978-0-444-63428-3.50310-6>.

Glover, Fred, and Gary A Kochenberger. Handbook Of Metaheuristics. Kluwer Academic Publishers, 2003.

Kirkpatrick, S., Gelatt, C., & Vecchi, M. (1983). Optimization by Simulated Annealing. *Science*, 220(4598), 671-680. Retrieved November 25, 2020, from <http://www.jstor.org/stable/1690046>

Brief review of static optimization methods, Editor(s): Stanisław Sieniutycz, Jacek Jeżowski, Energy Optimization in Process Systems and Fuel Cells (Third Edition), Elsevier, 2018, Pages 1-41, ISBN 9780081025574, <https://doi.org/10.1016/B978-0-08-102557-4.00001-3>.

NIU, M., WAN, C. & Xu, Z. A review on applications of heuristic optimization algorithms for optimal power flow in modern power systems. *J. Mod. Power Syst. Clean Energy* 2, 289–297 (2014), <https://doi.org/10.1007/s40565-014-0089-4>

J. L. Wang, Y. H. Lin and M. D. Lin, "Application of heuristic algorithms on groundwater pumping source identification problems," 2015 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM), Singapore, 2015, pp. 858-862, <https://doi.org/10.1109/IEEM.2015.7385770>.

Matott, L. Shawn, et al. "Application of Heuristic Optimization Techniques and Algorithm Tuning to Multilayered Sorptive Barrier Design." *Environmental Science & Technology*, vol. 40, no. 20, 2006, pp. 6354–6360., <https://doi.org/10.1021/es052560+>.

Larranaga P, Calvo B, Santana R, Bielza C, Galdiano J, Inza I, Lozano JA, Armananzas R, Santafe G, Perez A, Robles V (2006) Machine learning in bioinformatics. *Brief Bioinform* 7(1):86–112